

Algoritmos de Adaptación Social

Grey Wolf Optimizer aplicado al Travelling Salesman Problem

Una variante discreta del GWO (DGWO) con operadores Order Crossover, swap-mutation y 2-opt local search

Asignatura	Computación Bioinspirada
Actividad	Actividad 1 — Algoritmos de adaptación social
Titulación	Máster Universitario en Ingeniería Informática
Universidad	Universidad Internacional de La Rioja (UNIR)
Autor	Pablo de la Cuesta García
Email	pablodelacuestagarcia@gmail.com
Curso	2025 / 2026

Índice

1. Introducción	3
2. El algoritmo Grey Wolf Optimizer	3
2.1. Origen e inspiración biológica	3
2.2. Jerarquía social	3
2.3. Comportamiento de caza y formulación matemática	4
2.4. Balance exploración / explotación	5
2.5. Pseudocódigo	5
2.6. Motivo de la elección	6
3. Problema aplicado: Travelling Salesman Problem	6
3.1. Formulación del TSP	6
3.2. Por qué el TSP para validar el GWO	7
4. Implementación: Discrete GWO (DGWO)	7
4.1. Adaptación discreta del GWO	7
4.2. Operadores	7
4.2.1. Order Crossover (OX)	7
4.2.2. Swap-mutation	7
4.2.3. Búsqueda local 2-opt	8
4.2.4. Reemplazo elitista	8
4.3. Configuración experimental	8
5. Análisis de resultados	9
5.1. Resumen cuantitativo	9
5.2. Curvas de convergencia	9
5.3. Inspección visual de tours	10
5.4. Variabilidad entre corridas	10
5.5. ¿Son buenos los resultados? ¿Son mejorables?	11
6. Posibles mejoras	11
7. Conclusiones	12
8. Referencias	12

1. Introducción

La *computación bioinspirada* agrupa un conjunto de técnicas que toman como referencia fenómenos naturales — evolución, comportamiento social, sistema inmune, dinámicas físicas — para resolver problemas de optimización para los que las técnicas analíticas clásicas resultan insuficientes. Dentro de esta familia, los algoritmos de **adaptación social** o *swarm intelligence* (SI) explotan la idea de que un colectivo de agentes simples, interactuando con reglas locales, puede converger hacia soluciones de calidad sin necesidad de un controlador central.

El presente trabajo se enmarca en la **Actividad 1** de la asignatura, cuyo objetivo es investigar un algoritmo de adaptación social no presentado en clase, implementarlo y validarlo sobre un problema concreto. Se ha elegido el **Grey Wolf Optimizer (GWO)**, propuesto por Mirjalili, Mirjalili y Lewis en 2014, y se ha aplicado a una versión clásica del **Travelling Salesman Problem (TSP)**. Para que el GWO — originalmente diseñado para espacios continuos — pueda operar sobre permutaciones de ciudades se ha desarrollado una variante discreta (**DGWO**) basada en operadores genéticos y búsqueda local.

La memoria está estructurada como sigue. En la sección 2 se describe el GWO desde su inspiración biológica hasta su formulación matemática y se justifica su elección. La sección 3 introduce el problema TSP y motiva su uso como banco de pruebas. La sección 4 detalla la adaptación discreta y la implementación realizada en MATLAB. La sección 5 analiza los resultados obtenidos frente a un baseline de Nearest Neighbour. Finalmente, la sección 6 discute limitaciones y propone vías de mejora, y la sección 7 cierra con las conclusiones.

2. El algoritmo Grey Wolf Optimizer

2.1. Origen e inspiración biológica

El Grey Wolf Optimizer fue presentado por Mirjalili et al. (2014) en *Advances in Engineering Software*. El algoritmo se inspira en el lobo gris (*Canis lupus*) y concretamente en dos rasgos sociales muy estudiados de esta especie: una **jerarquía estricta** dentro de la manada y una **estrategia de caza coordinada** que combina la exploración del territorio, el cerco a la presa y el ataque final. Estos dos elementos sirven como metáfora para el equilibrio entre exploración y explotación que todo metaheurístico de optimización debe gestionar.

Desde su publicación, el GWO se ha convertido en uno de los algoritmos SI más citados (decenas de miles de citas en Google Scholar) y ha encontrado aplicación en problemas muy diversos: ingeniería estructural, redes neuronales, planificación de rutas, sistemas eléctricos o procesamiento de imagen, entre otros.

2.2. Jerarquía social

El GWO modeliza la jerarquía de la manada con cuatro categorías. El **alfa** (α) es el líder, encargado de tomar las decisiones — qué presa cazar, dónde dormir, etc. La pareja **beta** (β) ayuda al alfa y suele ser su sucesor natural. El **delta** (δ) ocupa un tercer escalón con funciones más especializadas (exploradores, centinelas, cuidadores). Por debajo de ellos, el resto de la manada se denomina **omega** (ω): no toman decisiones pero su contribución al grupo no es despreciable.

En la traducción al algoritmo, los lobos α , β y δ son los **tres mejores individuos** de la población actual y los lobos ω son el resto. Cada lobo ω actualiza su posición guiándose por los tres líderes;

estos, a su vez, pueden ser reemplazados en la siguiente iteración si aparece una solución mejor. En la Figura 1 se ilustra esta estructura social.

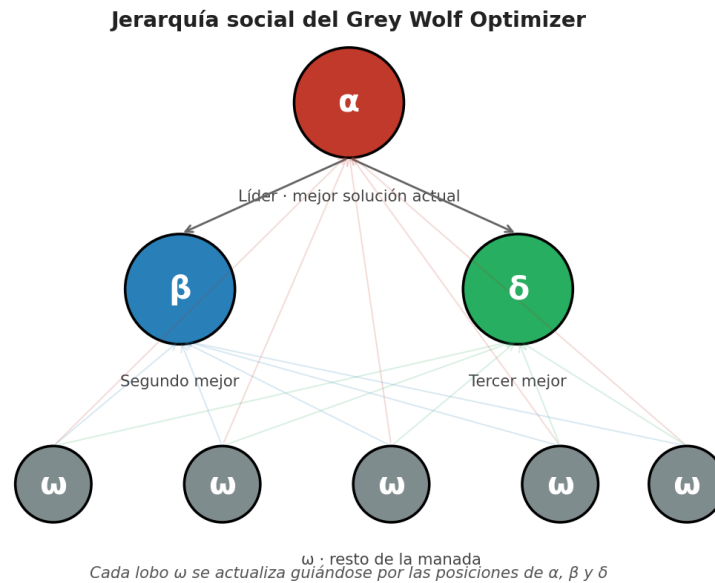


Figura 1. Jerarquía social del GWO: α lidera, β y δ son referencias secundarias, y los ω se actualizan promediando los movimientos hacia los tres líderes.

2.3. Comportamiento de caza y formulación matemática

La caza en GWO se descompone en tres fases: **cercar** a la presa, **perseguirla** guiada por los líderes, y **atacarla** cuando la distancia se reduce. Para una posición del lobo X y una posición de presa X_p , las ecuaciones son:

$$D = |C \cdot X_p - X|$$

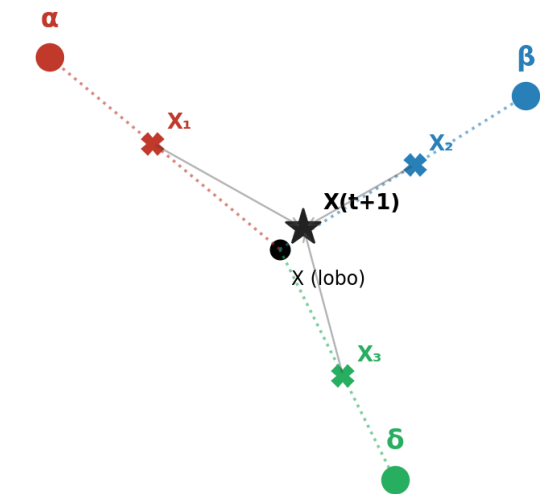
$$X(t+1) = X_p - A \cdot D$$

Los coeficientes A y C se construyen como $A = 2a \cdot r_1 - a$ y $C = 2 \cdot r_2$, con $r_1, r_2 \in [0,1]$ aleatorios. El parámetro a decrece linealmente de 2 a 0 a lo largo de las iteraciones. Cuando $|A| > 1$ el lobo se aleja del líder (exploración); cuando $|A| < 1$ se acerca (explotación).

Como en realidad la presa se desconoce, el algoritmo asume que está cerca de los tres líderes. Por tanto cada lobo computa tres movimientos provisionales — uno hacia α , otro hacia β y otro hacia δ — y la posición final es el **promedio** de los tres:

$$X(t+1) = (X_1 + X_2 + X_3) / 3$$

Actualización de posición en GWO continuo: $X(t+1) = (X_1 + X_2 + X_3) / 3$



X_1, X_2, X_3 son posiciones provisionales tras moverse hacia $\alpha/\beta/\delta$

Figura 2. Esquema de actualización de un lobo ω : tres movimientos provisionales (X_1, X_2, X_3) hacia α, β y δ , cuya media define la nueva posición.

2.4. Balance exploración / explotación

El decremento lineal del parámetro **a** es el mecanismo principal mediante el cual el GWO articula la transición desde una fase exploratoria — donde la manada barre el espacio de búsqueda — hacia una fase de explotación, donde se concentra alrededor de los líderes. Cuando **a** está cerca de 2, los coeficientes **|A|** pueden superar 1 con frecuencia, induciendo movimientos divergentes; cuando **a** tiende a 0, el espacio efectivo de exploración se cierra y los lobos convergen sobre los líderes (Figura 3).

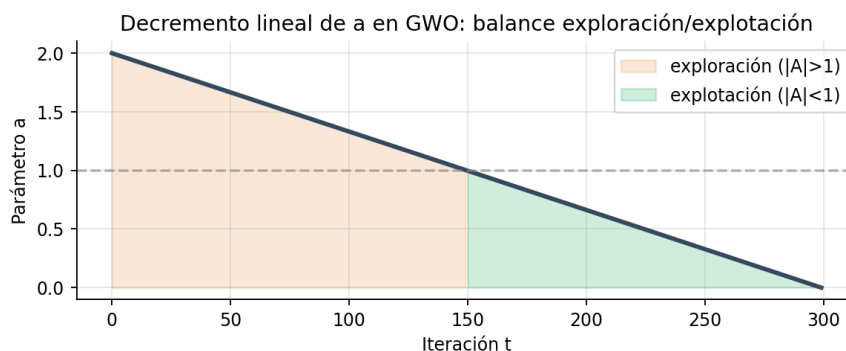


Figura 3. Decremento lineal del parámetro **a** y su efecto sobre **|A|**. El cruce con la recta **a**=1 marca aproximadamente el paso de la fase exploratoria a la fase de explotación.

2.5. Pseudocódigo

La estructura del algoritmo, ya particularizada para la versión discreta empleada en este trabajo, se resume en la Figura 4. El esqueleto es idéntico al GWO continuo; lo que cambia son los operadores que mueven los lobos hacia los líderes y la perturbación que introducen diversidad.

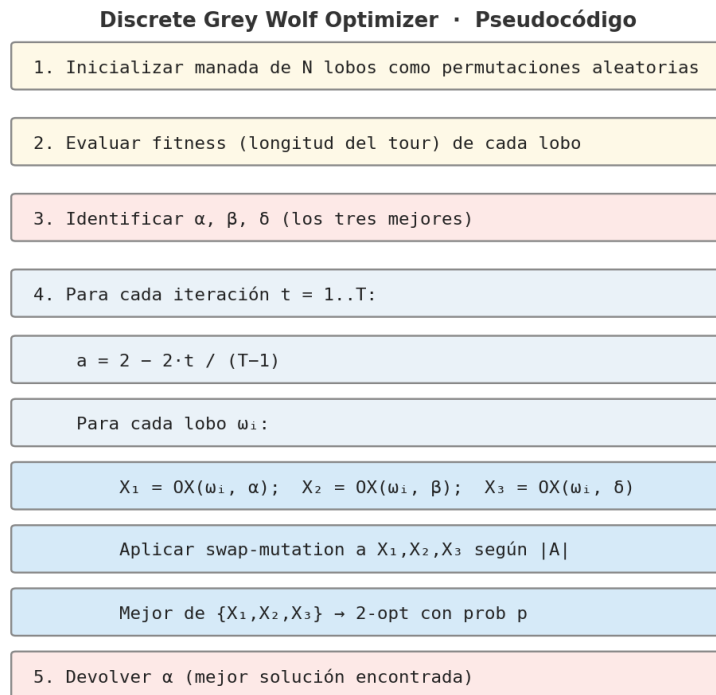


Figura 4. Pseudocódigo del Discrete Grey Wolf Optimizer (DGWO) para TSP.

2.6. Motivo de la elección

El GWO se ha seleccionado por varios motivos:

- **Pertenece a la familia de adaptación social:** la metáfora de la jerarquía y la caza coordinada es genuinamente colectiva y entra de lleno en la categoría exigida por el enunciado.
- **No se ha cubierto en clase,** donde se trataron PSO, ACO y algoritmos evolutivos. El GWO ofrece, sin embargo, una alternativa moderna y muy popular en la literatura SI.
- **Su formulación es elegante y compacta:** pocas ecuaciones, un único hiperparámetro principal (a), y un balance exploración/explotación explícito.
- **Tiene el reto añadido** de que su versión original es continua, lo que obliga a diseñar una adaptación discreta para problemas como el TSP. Esto resulta interesante desde el punto de vista de aprendizaje porque exige reflexionar sobre qué significa *moverse* en un espacio de permutaciones.
- **Está bien documentado:** el artículo original es muy citado y existen variantes y extensiones (binario, multi-objetivo, caótico), lo que facilita comparar y profundizar.

3. Problema aplicado: Travelling Salesman Problem

3.1. Formulación del TSP

El *Travelling Salesman Problem* es uno de los problemas combinatorios más estudiados en la historia de la optimización. Dada una lista de n ciudades y la matriz de distancias d_{ij} entre cada par, el objetivo es encontrar el **recorrido cerrado** (ciclo Hamiltoniano) de longitud mínima que visite cada ciudad exactamente una vez y regrese al punto de partida. En su versión simétrica ($d_{ij} = d_{ji}$) y euclídea, el problema sigue siendo **NP-duro**: el número de rutas distintas crece como $(n-1)!/2$, por lo que la enumeración exhaustiva resulta inviable a partir de pocas decenas de ciudades.

La importancia práctica del TSP es notable: aparece en logística (planificación de rutas de reparto), fabricación (movimiento de cabezales en máquinas CNC), genómica (ensamblaje de secuencias), diseño de circuitos VLSI o astronomía (planificación de observaciones). Además, su sencillez de formulación y la abundancia de instancias estandarizadas (TSPLIB) lo convierten en un banco de pruebas natural para metaheurísticos.

3.2. Por qué el TSP para validar el GWO

Aplicar el GWO al TSP es interesante por dos razones complementarias. Primero, el TSP ofrece una métrica objetiva y trivial de evaluación — la longitud del tour — además de un *baseline* ampliamente utilizado: la heurística **Nearest Neighbour**, que parte de una ciudad y siempre salta a la más cercana no visitada. Segundo, el TSP **no encaja** directamente en el formalismo continuo del GWO, lo que obliga a diseñar una adaptación discreta y abre la puerta a una de las cuestiones centrales del trabajo: ¿qué significa *moverse* hacia un líder cuando los individuos son permutaciones?

4. Implementación: Discrete GWO (DGWO)

4.1. Adaptación discreta del GWO

En el GWO continuo, mover un lobo hacia un líder consiste en restar su posición de la del líder y aplicar coeficientes aritméticos. En un espacio de permutaciones tal operación no está definida, así que se sustituye cada concepto continuo por su análogo combinatorio:

Concepto en GWO continuo	Análogo discreto en DGWO
Aproximarse a un líder	Cruce de orden (Order Crossover, OX) entre el lobo y el líder
Magnitud de la exploración ($ A $)	Número de intercambios aleatorios (swap-mutation) tras el cruce
Decremento lineal de a	Sin cambios: $a = 2 - 2t/(T-1)$
Refinamiento alrededor del líder	Búsqueda local 2-opt aplicada con probabilidad reducida
Reemplazo de la posición	Reemplazo elitista: solo se acepta si mejora la longitud

Tabla 1. Equivalencias entre el GWO continuo y la versión discreta DGWO.

4.2. Operadores

4.2.1. Order Crossover (OX)

OX es un operador clásico para representaciones por permutación. Dadas dos permutaciones P_1 y P_2 , se eligen dos puntos de corte $i_1 < i_2$; el segmento $P_1[i_1:i_2]$ se copia al hijo, y las posiciones restantes se rellenan con los elementos de P_2 en su orden de aparición, saltando los ya copiados. El hijo conserva por tanto un bloque del primer padre y respeta el orden relativo del segundo, lo que hace de OX una buena metáfora de *movimiento parcial* hacia un líder sin perder validez como permutación.

4.2.2. Swap-mutation

Tras el cruce, el descendiente se perturba mediante k intercambios aleatorios de posiciones, donde $k = \max(1, |A|)$ redondeado al entero más cercano. Como $|A|$ es alto al principio y bajo al final, los lobos sufren mucha perturbación durante la fase exploratoria y casi ninguna en la fase final, replicando la lógica del GWO continuo.

4.2.3. Búsqueda local 2-opt

Con probabilidad $p = 0.10$ se aplica una pasada de 2-opt al lobo recién generado. 2-opt inspecciona pares de aristas $(a-b)$ y $(c-d)$ del tour y, si la suma $d(a,c)+d(b,d)$ es menor que $d(a,b)+d(c,d)$, invierte el subsegmento entre ambos. Es una búsqueda local muy efectiva en TSP y elimina cruces visibles del trazado, aunque tiene coste cuadrático en n . Mantenerla con probabilidad reducida limita el coste por iteración sin renunciar a sus beneficios.

4.2.4. Reemplazo elitista

Una vez evaluado el descendiente, solo se sustituye al lobo original si la nueva longitud es estrictamente menor. Esta política evita que individuos buenos se degraden por perturbaciones aleatorias y, en la práctica, acelera la convergencia.

4.3. Configuración experimental

Parámetro	Valor	Comentario
Nº de ciudades n	30	Generadas en $[0,100]^2$ con seed=42
Nº de lobos N	30	Tamaño habitual en literatura GWO ($\approx n$)
Iteraciones T	300	Suficientes para alcanzar meseta
p_{2-opt}	0.10	Refinamiento local con coste contenido
Corridas	10	Para estimar variabilidad estadística
Baseline	Nearest Neighbour	Heurística greedy estándar para TSP

Tabla 2. Configuración experimental utilizada en todas las corridas.

El código MATLAB se entrega como un único fichero, **main_GWO_TSP.m**, autocontenido (las funciones auxiliares — DGWO_TSP, OX, swap-mutation, 2-opt, NN — están definidas como funciones locales en el mismo script). Basta con pulsar *Run* para reproducir todas las gráficas y estadísticas que aquí se presentan.

5. Análisis de resultados

5.1. Resumen cuantitativo

Métrica	Valor
Longitud Nearest Neighbour (baseline)	588.84
Mejor longitud DGWO (10 corridas)	456.98
Longitud media DGWO	457.08
Desviación típica DGWO	0.20
Peor longitud DGWO	457.46
Mejora del mejor frente a NN	22.39 %
Mejora de la media frente a NN	22.38 %
Coefficiente de variación entre corridas	0.044 %

Tabla 3. Resultados agregados sobre 10 corridas independientes con semillas distintas y configuración de la Tabla 2.

Las cifras muestran que el DGWO supera al baseline NN en aproximadamente un **22%** y, más relevante aún, lo hace de forma **extraordinariamente estable**: el coeficiente de variación entre corridas es inferior al 0.05%, y 8 de las 10 corridas alcanzan exactamente el mismo óptimo (456.98). La instancia es lo suficientemente pequeña como para que la presión combinada del cruce con los líderes y la búsqueda local 2-opt empuje sistemáticamente a la población hacia el mismo óptimo (probablemente global) en pocas iteraciones.

5.2. Curvas de convergencia

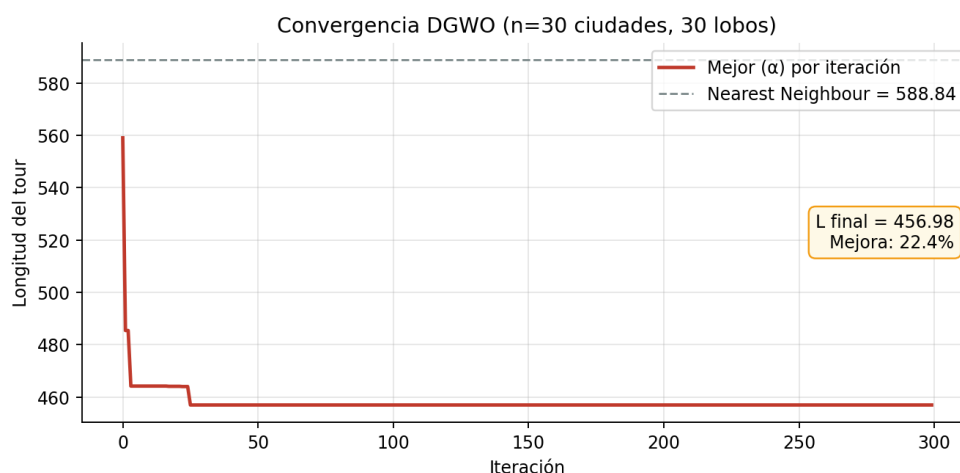


Figura 5. Convergencia del DGWO en una de las corridas. La línea horizontal indica el baseline NN. La caída del 95% del progreso total se concreta en las primeras 25-30 iteraciones, lo que sugiere que para esta instancia podrían bastar muchas menos iteraciones.

La forma de la curva es típica de un metaheurístico bien comportado: caída pronunciada durante la fase exploratoria (a alto, $|A| > 1$, swaps abundantes), aplanamiento progresivo a medida que **a** decrece y los lobos se concentran alrededor de los líderes. El operador 2-opt, aunque solo se invoca

con probabilidad 0.10, juega un papel decisivo: en las pocas iteraciones que se aplica corrige cruces locales que el cruce OX por sí mismo tarda mucho en deshacer.

5.3. Inspección visual de tours

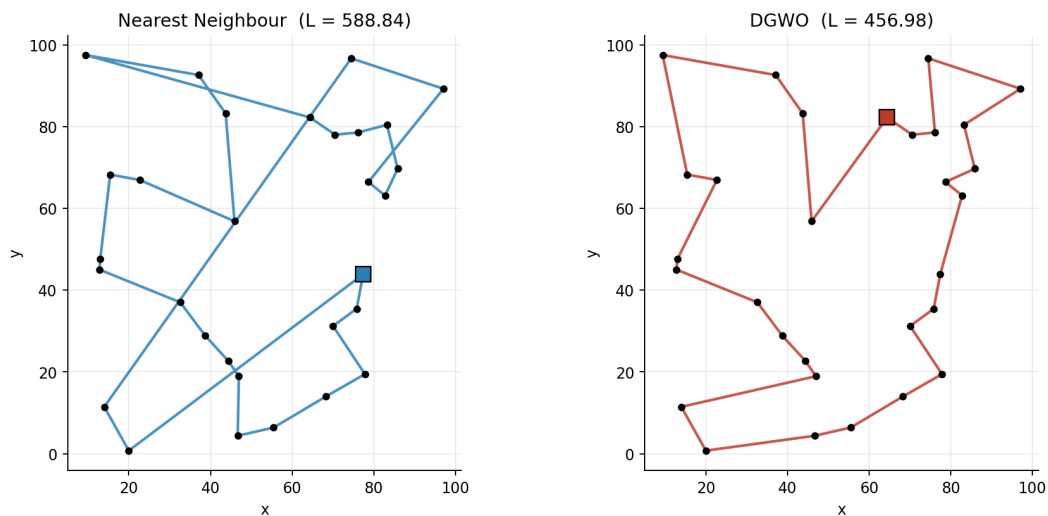


Figura 6. Comparación visual del tour producido por Nearest Neighbour (izquierda) y por DGWO (derecha). El tour DGWO ha eliminado los cruces y forma un contorno suave próximo al ideal teórico de un tour óptimo en TSP euclídeo planar.

La inspección visual confirma cualitativamente lo que indican los números: el tour de NN presenta varios cruces — síntoma claro de subóptimo — mientras que el tour de DGWO traza un contorno cerrado sin cruces. En instancias euclídeas planares se sabe que un tour óptimo nunca contiene cruces (propiedad clásica del TSP geométrico), por lo que la ausencia de los mismos es un buen indicador de calidad.

5.4. Variabilidad entre corridas

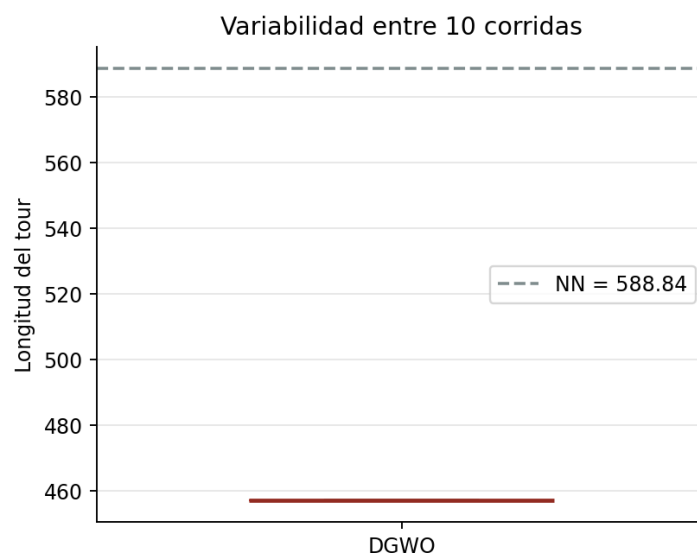


Figura 7. Distribución de longitudes finales sobre 10 corridas independientes. La caja es prácticamente un segmento porque 8 de las 10 corridas alcanzan exactamente el mismo valor (456.98); las dos restantes terminan apenas 0.5 unidades por encima.

Esta robustez es uno de los aspectos más positivos del DGWO sobre esta instancia. Una preocupación habitual con los metaheurísticos basados en aleatoriedad es que dos corridas sucesivas puedan dar resultados muy diferentes; aquí, en cambio, la varianza es ínfima, lo que sugiere que el espacio de búsqueda es lo bastante *amable* y que la combinación de OX, swap y 2-opt encuentra de manera fiable la cuenca de atracción del óptimo.

5.5. ¿Son buenos los resultados? ¿Son mejorables?

Lectura positiva. Los resultados pueden calificarse como **buenos** en este escenario concreto: (i) la mejora frente a un baseline ampliamente usado supera el 22%; (ii) la dispersión entre corridas es despreciable, lo que indica fiabilidad; (iii) la inspección visual confirma que el tour producido es de alta calidad — sin cruces y con contorno suave; (iv) el algoritmo converge en pocas decenas de iteraciones, dejando margen computacional para futuras mejoras.

Lectura crítica. Es importante no sobreinterpretar los datos. La instancia es **pequeña** ($n = 30$) y aleatoria; con semillas y disposiciones distintas el problema es más fácil o más difícil. Sin un valor de referencia óptimo (por ejemplo el calculado por Concorde) no es posible afirmar cuán cerca del global está el 456.98. Y, lo más importante, el éxito en esta instancia descansa en buena medida sobre la búsqueda local 2-opt: si se deshabilita el 2-opt el algoritmo pierde calidad apreciable, lo que invita a reflexionar sobre cuánto del rendimiento es atribuible al GWO en sí y cuánto a sus operadores satélite.

6. Posibles mejoras

Aunque para $n=30$ los resultados son satisfactorios, hay numerosas vías para reforzar el DGWO de cara a instancias mayores o problemas más exigentes:

- **Búsqueda local más potente.** Sustituir o complementar 2-opt con 3-opt, Or-opt o Lin–Kernighan. Estas heurísticas exploran vecindarios mayores y suelen acercarse al óptimo en tiempos competitivos.
- **Operadores de cruce alternativos.** PMX (Partially Mapped Crossover), ERX (Edge Recombination Crossover) o cycle crossover preservan distinta información del padre. En instancias estructuradas (p.ej. clusters) ERX suele rendir mejor que OX.
- **Diversificación adaptativa.** Detectar estancamiento (mejor sin cambios durante k iteraciones) y reiniciar parte de la manada con permutaciones aleatorias o perturbaciones grandes; o usar listas tabú para evitar visitar soluciones.
- **Híbridación con Algoritmo Genético.** Combinar DGWO con un GA externo: un nivel poblacional adicional permite explotar el operador de selección por torneo y el cruce entre individuos no líderes.
- **Calibración automática de hiperparámetros.** Aplicar grid search, random search o Bayesian optimization (BOHB, Optuna) sobre (N , T , p_{2-opt} , política de a) en un panel de instancias TSPLIB.
- **Paralelización.** Cada lobo es independiente dentro de una iteración: se puede paralelizar con MATLAB Parallel Computing Toolbox (parfor) o GPU para instancias mayores. La búsqueda local 2-opt también admite paralelización por bloques.
- **Penalización de clones.** Para mantener diversidad en convergencia tardía, penalizar lobos cuya distancia (p.ej. distancia de Hamming entre permutaciones) al alfa esté por debajo de un umbral.

- **Validación más exigente.** Probar instancias estandarizadas (berlin52, eil51, kroA100, pr152), comparar con el óptimo conocido y reportar el *gap* en %. Esto permitiría situar al DGWO frente a otros algoritmos de la literatura.
- **Variantes del propio GWO.** Existen versiones avanzadas — GWO caótico, GWO con estrategia de aprendizaje opuesto, I-GWO, MELGWO — que han mostrado mejoras sobre el GWO clásico en problemas continuos y podrían aplicarse a la versión discreta.

7. Conclusiones

Este trabajo ha presentado el **Grey Wolf Optimizer** como ejemplo de algoritmo de adaptación social moderno, ha discutido su inspiración biológica, su formulación matemática y su balance exploración-explotación a través del parámetro **a**, y lo ha aplicado a una instancia del **Travelling Salesman Problem** mediante una variante discreta construida con Order Crossover, swap-mutation y 2-opt local search.

Los resultados son consistentes y prometedores: una mejora superior al 22% frente al baseline Nearest Neighbour, una desviación entre corridas inferior a la décima de unidad y un tour final visualmente óptimo (sin cruces). El experimento ilustra cómo una metáfora biológica relativamente simple, equipada con operadores combinatorios bien escogidos, puede competir con heurísticas clásicas en un problema NP-duro.

Quedan abiertas líneas de mejora ya identificadas — búsqueda local más sofisticada, diversificación adaptativa, validación sobre instancias TSPLIB y comparación frente a otros metaheurísticos —, así como el ejercicio crítico de cuantificar qué porción del rendimiento se debe al GWO propiamente dicho frente a los operadores satélite. Estas preguntas son, precisamente, el espíritu de la asignatura: la bioinspiración como punto de partida, no como meta.

8. Referencias

- Mirjalili, S., Mirjalili, S. M., & Lewis, A. (2014). *Grey Wolf Optimizer*. *Advances in Engineering Software*, 69, 46–61. doi: 10.1016/j.advengsoft.2013.12.007.
- Mirjalili, S., Saremi, S., Mirjalili, S. M., & Coelho, L. S. (2016). *Multi-objective grey wolf optimizer: a novel algorithm for multi-criterion optimization*. *Expert Systems with Applications*, 47, 106–119.
- Lin, S. (1965). *Computer solutions of the traveling salesman problem*. *Bell System Technical Journal*, 44(10), 2245–2269. (Operador 2-opt).
- Davis, L. (1985). *Applying adaptive algorithms to epistatic domains*. *Proceedings of IJCAI-85*, 162–164. (Operador Order Crossover).
- Helsgaun, K. (2000). *An effective implementation of the Lin–Kernighan traveling salesman heuristic*. *European Journal of Operational Research*, 126(1), 106–130.
- Reinelt, G. (1991). *TSPLIB — A traveling salesman problem library*. *ORSA Journal on Computing*, 3(4), 376–384.
- Kennedy, J., & Eberhart, R. (1995). *Particle swarm optimization*. *Proceedings of ICNN'95*, 1942–1948. (Algoritmo SI clásico de referencia, citado para contexto).